

Django 課程先備

Python 語法先備

給「只學過一點或沒學過 Python」就要開始 Django 的你。

目標不是精通 Python，而是讀得懂接下來的兩個專案（LearnBoard 留言板、LearnMart 商城）的每一行程式碼。

本課程的兩個專案：本冊為兩者共用

階段	專案	主要 Python 程式碼位置
第一階段	LearnBoard 學言板 (<code>learnboard/</code>)	<code>board/models.py</code> 、 <code>board/views.py</code>
第二階段	LearnMart 學購商城 (<code>learnmart/</code>)	<code>marketplace/models.py</code> 、 <code>marketplace/views.py</code>

- 本冊語法在兩個專案都會出現；留言板的程式碼比商城精簡，適合第一次對照
- 內文標有「**LearnMart 對照**」的例子屬於第二階段的伏筆，第一階段可先跳過或當預習
- 看到任一專案的 `view` / `model` 程式碼，你能逐行說出它在做什麼

這份教材解決什麼問題？

Django 官方教學假設你已經會 Python。但本課程兩個專案的程式碼會用到：

- 基礎語法(變數、if、for...)、f-string、dict、`None`、truthy/falsy (第 1~4 章)
- `def`、預設值、`*args` / `**kwargs` (第 5~6 章)
- `try` / `except`、import (第 7~8 章)
- class、繼承、`super()`、mixin、decorator、型別標註 (第 9~12 章)

每章都會指出「這個語法出現在課程專案哪個檔案」，學了立刻用得上。

判斷標準：看到 `board/views.py` 或 `marketplace/views.py` 的程式碼，逐行說出它在做什么。

練習環境：兩種方式都夠用

方式一：互動直譯器（REPL）——適合本章所有小實驗

```
uv run python
```

```
>>> 1 + 2
```

```
3
```

```
>>> exit()
```

- `>>>` 是提示符：輸入一行、立刻看結果
- 離開用 `exit()`

方式二：寫成檔案再執行

```
# 把程式存進 scratch.py 後：  
uv run python scratch.py
```

十三章地圖：由「資料」走到「物件」

章	主題	在課程專案最先看到	章	主題	在課程專案最先看到
1	變數與基本型別	<code>price = Decimal("350")</code>	8	import	<code>from .models import Product</code>
2	字串與 f-string	<code>messages.success(f"...")</code>	9	類別基礎	Model class、 <code>__str__</code>
3	list / tuple / dict / set	template context、 <code>choices</code>	10	繼承與 mixin	<code>User(AbstractUser)</code> 、CBV mixin
4	流程控制與真偽值	<code>if request.method == "POST"</code>	11	decorator	<code>@login_required</code>
5	迴圈	<code>{% for %}</code> 背後的 for	12	型別標註	<code>-> QuerySet</code>
6	函式 <code>def</code>	view function、 <code>*args/**kwargs</code>	13	對照總表與自我檢查	—
7	例外處理	<code>int(request.POST.get(...))</code>			

每章結構：概念 → 最小範例 → 「動手試」 → 觀念檢核（附簡答）。

第 1 章

變數與基本型別

本章成果： 能建立變數、分辨四個基本型別與 `None`，並讀懂賦值陳述句。

1-1 變數：替「值」取名字

```
product_name = "機械鍵盤"  
stock = 12
```

- `=` 是**綁定**：把右邊的值和左邊的名字連起來
- 不是數學等號：`stock = stock + 1` 表示「拿舊值加 1，重新綁回 stock」
- 命名慣例：小寫加底線 `snake_case`，例如 `order_items`

常見錯誤：使用尚未建立的變數 → `NameError: name 'prodect' is not defined`。九成的拼字錯誤都會變成 `NameError`。

1-2 四個基本型別

型別	字面寫法	用途	課程專案例子
int	12	整數	stock = 0
float	12.5	小數（金額不用它！）	統計平均值
str	"鍵盤"	文字	商品名稱
bool	True / False	真偽	is_active = True

用 `type()` 查詢型別：

```
>>> type(12)
<class 'int'>
>>> type("鍵盤")
<class 'str'>
```

注意：金額用 `Decimal` 不用 `float` 。

專案盤點：金額型別

兩個專案的盤點：目前 LearnBoard／LearnMart 的 Python 原始碼沒有把金額寫成 `float`。LearnMart 的訂單金額欄位使用 `DecimalField`，預設值以 `Decimal("0")` 建立；`price`、`total` 這類欄位不要自行改成 `float`。

```
# learnmart/marketplace/models.py
from decimal import Decimal

total = models.DecimalField(..., default=Decimal("0"))
```

1-3 None：「這裡故意沒有值」

```
result = None          # 先宣告，稍後才決定

def find_product(pk):
    ...
    return None        # 找不到時明確回傳「沒有」
```

- `None` 是一個值，代表「無」；列印出來是 `None` 不是空字串
- 判斷要用 `is`：

```
if average is None:
    print("還沒有任何評分")
```

LearnMart 對照： 商品沒有評價時，`average_rating` property 回傳 `None`，模板據此顯示「尚無評分」。

1-4 動態型別：名字不綁定型別

```
value = 42          # value 綁到 int
value = "42"        # 同一個名字改綁 str，合法但不一定明智
```

- Python 變數沒有固定型別；型別跟著「值」走
- `"42"` 是文字，`"42" + 1` 會直接 `TypeError`

你應該看到：

```
TypeError: can only concatenate str (not "int") to str
```

讀錯誤訊息的順序：先看最後一行的型別與訊息，再往上看是哪一行觸發。

1-5 print、註解與一行多重綁定

```
a, b = 3, 5          # 一行綁定兩個名字
print(a, b)          # 3 5

# 井字號之後是註解，Python 不會執行
total = a + b         # 這種行內註解說明「為什麼」最有價值
print(total)
```

- `print()` 可以一次印多個值，以空白分隔
- 註解寫「為什麼這樣寫」，不要逐句翻譯程式本身

1-6 從 LearnBoard 看「名字綁到值」

= 右邊不一定是數字或字串，也可能是 Django 物件；先把「名稱指向某個值」這個 Python 觀念讀懂即可。

```
# learnboard/board/views.py
queryset = Message.objects.select_related("author")
query = self.request.GET.get("q", "").strip()
```

```
# learnboard/board/views.py
context["query"] = self.request.GET.get("q", "")
```

- `queryset` 綁到查詢集合，`query` 綁到整理過的搜尋文字
- `"q"`、`""` 是 `str`；`paginate_by = 10` 則是 `int`
- `context["query"] = ...` 是先取得 dict 的 key，再把值放進去

這些變數的型別可以不同；重點是先辨認「名稱、右側值，以及右側值接下來能做什麼」。

1-7 兩個專案都用 `None` 表示「沒有關聯資料」

LearnBoard 的留言作者允許沒有值；LearnMart 的商品沒有任何評價時，平均分數也故意回傳 `None`。

```
# learnboard/board/models.py
who = self.author.username if self.author else "訪客"

# learnmart/marketplace/models.py
result = self.reviews.aggregate(avg=models.Avg("rating"))["avg"]
return round(result, 1) if result else None
```

- LearnBoard 的 `author` 設定 `null=True`，所以可能沒有登入作者
- LearnMart 的 `result` 沒有評價時沒有平均值，回傳 `None`，不是 `0`
- `if ... else ...` 是條件運算式；完整的真假值與分支會在第 4 章整理

讀程式時的問題： 這個名稱可能沒有值嗎？如果沒有，畫面應該顯示什麼？

1-8 一行多重綁定：專案常用的回傳值拆開

Django 的方法常回傳兩個值，Python 可以在同一行分別綁定：

```
# learnboard/board/management/commands/seed_demo.py
user, created = User.objects.get_or_create(...)

# learnmart/marketplace/views.py
item, created = CartItem.objects.get_or_create(
    user=request.user, product=product
)
```

- `user` / `item` 是取得或建立的物件
- `created` 是 `bool`：這次是否真的新建立
- 這不是把兩個東西塞進一個變數，而是把回傳的兩個位置拆開

同樣的寫法也出現在 `for username, email in [...]`；左邊兩個名稱會依序接住每筆資料的兩個欄位。

第 1 章 | 動手試與觀念檢核

動手試 (REPL)： 預測每一行輸出，再實際執行：

```
x = 10
y = x
x = 20
print(y)           # ?
type(None)         # ?
print("5" + 5)      # ? (先想，再執行看錯誤)
```

觀念檢核：

1. `stock = stock - 1` 若 stock 原本是 3，執行後是多少？ **答：2**
2. `None` 和空字串 `""` 一樣嗎？ **答：不同；None 代表沒有值，"" 是長度 0 的文字**
3. 怎麼安全判斷「是不是 None」？ **答：if value is None**
4. `nameerror` 和 `NameError` 是同一個名字嗎？ **答：不是；Python 大小寫敏感**

第 2 章

字串與 f-string

本章成果： 能串接、格式化字串，並使用 `strip()`、`split()` 等高頻方法。

2-1 建立字串：三種引號

```
name = "機械鍵盤"      # 雙引號
title = '學購商城'     # 單引號，等價
doc = """多行文字
可以換行"""           # 三引號：保留換行
```

- 引號要成對；字串內含雙引號時，外層改用單引號
- 跳脫字元 `\n` 是換行、`\t` 是定位鍵

常見錯誤： `"It's fine"` 會把 `'` 當結束 → 改成 `'It's fine'` 寫法錯誤，應寫 `"It's fine"` 或 `"It\'s fine"`。

2-2 串接 vs f-string：一律優先 f-string

```
user = "leon"
count = 3

greeting = "Hi, " + user          # 可以，但遇到數字就麻煩
msg = "Hi, " + user + ", 你有 " + str(count) + " 件商品"

msg = f"Hi, {user}, 你有 {count} 件商品"  # 推薦
```

- f-string：字串前加 `f`，`{}` 內放任何 Python 運算式
- `{}` 內可以是方法呼叫：`f"{product.name.upper()}"`

專案對照：f-string

LearnMart 對照：

```
messages.success(request, f"已將「{product.name}」加入購物車。")
```

LearnBoard 對照：

```
# learnboard/board/models.py  
return f"{who}：{self.content[:20]}"
```

`{product.name}`、`{who}` 都是先取得值，再嵌入字串；大括號內也可以放屬性存取或其他 Python 運算式。

2-3 四個常用的字串方法與專案盤點

方法	作用	例子	兩個專案的實際情況
<code>.strip()</code>	去頭尾空白	<code>" q ".strip()</code> → <code>"q"</code>	LearnBoard、LearnMart 的搜尋都直接使用
<code>.split(",")</code>	依分隔符切 list	<code>"a,b".split(",")</code> → <code>["a", "b"]</code>	目前沒有直接呼叫
<code>",".join(xs)</code>	把 list 接回字串	<code>",".join(["a","b"])</code> → <code>"a,b"</code>	目前沒有直接呼叫
<code>.lower()</code>	將文字轉小寫	<code>"ABC".lower()</code> → <code>"abc"</code>	目前沒有直接呼叫；搜尋使用 Django 的 <code>icontains</code>

方法用點呼叫：`"字串".方法(參數)`。它們多半回傳新字串，不改原本的值。

`split()`、`join()`、`lower()` 仍然值得學，但不要把練習範例誤認成目前兩個專案已經存在的程式碼。

2-4 len、in 與切片

```
name = "LearnMart"

len(name)      # 9：字元數
"Mart" in name # True：包含子字串？
name[0]        # 'L'：索引從 0 開始
name[-1]       # 't'：負索引從尾巴數
name[:5]       # 'Learn'：切片 [起:止)，含頭不含尾
```

你應該看到： `name[5]` 是 `'M'`——因為第 0 位是 `'L'`。

常見錯誤： 索引超出範圍 → `IndexError: string index out of range`。

專案對照：切片與 `len()`

LearnBoard 對照：

```
# learnboard/board/models.py
return f"{who} : {self.content[:20]}"

# learnboard/board/admin.py
return obj.content[:30]
```

`[:20]` 與 `[:30]` 都是「從開頭取到指定位置之前」；留言太長時，管理介面與物件顯示文字只取前面一段。這裡的 `icontains` 則是 Django ORM 的查詢 lookup，不是 Python 的 `in` 運算子。

專案盤點：兩個專案的 Python 原始碼目前沒有直接呼叫 `len()`；模板裡的 `.count()`、分頁筆數等是 Django／模板 API，不等同於本頁的 Python `len()`。

2-5 LearnBoard：搜尋文字的完整路徑

```
# learnboard/board/views.py
query = self.request.GET.get("q", "").strip()
if query:
    queryset = queryset.filter(Q(content__icontains=query))
```

把這段拆成三步：

1. 從 GET 參數取出 `q`；沒有參數時使用空字串 `""`
2. 用 `strip()` 移除使用者不小心輸入的頭尾空白
3. 有搜尋文字時，交給 Django 的 `icontains` 查找留言內容

因此，使用者輸入 `" Django "` 後，查詢使用的是 `"Django"`。
`query` 是 Python 的字串；

`Q(content__icontains=query)` 則是 Django ORM 的查詢表達式。

2-6 LearnMart：同樣的整理方式，加上提示訊息

```
# learnmart/marketplace/views.py
query = self.request.GET.get("q", "").strip()
category = self.request.GET.get("category", "").strip()

messages.success(request, f"已將「{product.name}」加入購物車。")
```

- 搜尋文字與分類代稱都先 `strip()`，所以兩個輸入欄位的處理規則一致
- f-string 把商品物件的 `name` 放進使用者看得到的訊息
- 同一個 view 也使用 `f"「{item.product.name}」庫存不足，請調整數量。"` 等訊息

對照重點：字串方法處理輸入，f-string 處理輸出；兩者都只是普通 Python 運算式，Django 只是把它們放進 request／response 的工作流程。

第 2 章 | 動手試與觀念檢核

動手試： 把使用者輸入整理成搜尋字串：

```
raw = "  鍵盤  "  
query = raw.strip()  
print(f"搜尋：{query} ({len(query)} 個字) ")  
# 搜尋：鍵盤 (2 個字)
```

觀念檢核：

1. `f"{3 + 4}"` 的結果？ 答："7"（先求值再轉文字）
2. `"a b c".split(" ")` 回傳什麼型別？ 答：list，["a", "b", "c"]
3. 如何安全地把 `" x "` 變成 `"x"`？ 答：`s = s.strip()`
4. `len(None)` 會成功嗎？ 答：不會，TypeError

第 3 章

容器：list、tuple、dict、set

本章成果： 能選對容器裝資料，並讀懂課程專案的 context dict 與 choices tuple。

3-1 list：有順序、可增改

```
products = ["鍵盤", "筆記本", "滑鼠"]

products[0]          # '鍵盤' (索引從 0 開始)
products.append("螢幕") # 尾端新增
products.remove("筆記本") # 依值刪除
len(products)        # 3
"滑鼠" in products    # True
```

常見錯誤： `products[10]` → `IndexError`；刪除不存在的值 → `ValueError: list.remove(x): x not in list`。

3-2 tuple：唯讀的小組資料

```
role = ("buyer", "買家")          # 兩件事綁在一起，不可改
code, label = role                 # 可解包到兩個名字

class Role(models.TextChoices):
    BUYER = "buyer", "買家"        # ← 這其實是 tuple ("buyer", "買家")
```

- tuple 用小括號；建立後不能 append/remove
- 需要「固定成組」的欄位定義就用 tuple

LearnMart 對照： `models.TextChoices` 的每個成員、`path()` 的額外參數 tuple，都是唯讀成組的概念。

3-4 dict : key → value 的對照表

```
context = {  
    "products": ["鍵盤", "筆記本"],  
    "query": "鍵",  
}  
  
context["query"]          # '鍵'  
context["categories"] = [] # 新增一組 key-value
```

- key 通常是字串；value 可以是任何東西（包括另一個 dict/list）
- **Template context 就是 dict**：key 成為模板裡的變數名

```
{# render(request, "home.html", context) 之後： #}  
{% query %}    {% for p in products %} ... {% endfor %}
```

3-5 dict 取值：`[]` 與 `.get()` 的差別

```
params = {"q": "鍵盤"}

params["q"]           # '鍵盤'
params["category"]    # KeyError! (沒有這個 key)

params.get("category") # None (找不到也不爆炸)
params.get("category", "") # '' (附帶預設值)
```

LearnMart 對照：

```
query = self.request.GET.get("q", "").strip()
```

`request.GET` 是類 dict 物件；`.get(key, 預設)` 保證「沒填也拿得到空字串」，後續 `.strip()` 才不會炸。

常見錯誤：對可能不存在的 key 直接用 `[]` → `KeyError`。

3-3 巢狀：一串 dict 是最常見的形狀

```
products = [  
    {"name": "鍵盤", "price": 350},  
    {"name": "筆記本", "price": 60},  
]  
  
products[0]["name"]      # '鍵盤'：先索引再取 key
```

讀法由外而內：`products[0]` 先拿到第一個 dict，再用 `["name"]` 取值。

你應該看到：這正是「商品目錄」在記憶體中的長相；存進資料庫後，一行 row 就是一個 dict 的角色。

3-6 走訪 dict : keys 、 values 、 items

```
stock = {"鍵盤": 12, "筆記本": 40}

for name in stock:                # 預設走訪 key
    print(name)

for name, qty in stock.items():    # 同時拿 key 和 value
    print(f"{name} 剩 {qty} 个")
```

`.items()` 配兩個迴圈變數是最常用的寫法（解包又出現了）。

3-7 set：自動去重覆的無序集合

```
tags = {"3C", "熱銷", "3C"}

len(tags)                # 2：重複的自動消失
"熱銷" in tags            # True (查 membership 很快)

unique = set(["a", "b", "a"]) # 從 list 轉 set 去重 → {"a", "b"}
```

何時想到 set？ 題目出現「只要判斷在不在／不要重複」。

目前專案盤點： 兩個專案的 Python 原始碼沒有直接建立 `set`。LearnMart 的 `.distinct()` 是 Django ORM 要求資料庫去除重複列，不是 Python 的 `set`。

```
# learnmart/marketplace/views.py
return Order.objects.filter(items__seller=self.request.user).distinct().prefetch_related("items")
```

3-8 LearnBoard : View 用 dict 準備 Template context

```
# learnboard/board/views.py
context = super().get_context_data(**kwargs)
context["query"] = self.request.GET.get("q", "")
return context
```

- `context` 是一個 dict ; `"query"` 是 key , 搜尋文字是 value
- `context["query"] = ...` 可以新增 key , 也可以覆寫原本的 value
- `request.GET` 是類 dict 物件 ; `.get("q", "")` 找不到參數時回傳空字串

模板裡的 `{{ query }}` , 就是讀取這個 context dict 的 `"query"` value 。

3-9 LearnMart : list、tuple、dict 一起工作

```
# learnmart/marketplace/management/commands/seed_demo.py
categories = {}
products = [
    ("教學用機械鍵盤", "tech", 1590, 20, "練習商品詳情頁與購物車流程的示範鍵盤。"),
    # 其餘商品資料省略
]

for name, slug, price, stock, description in products:
    Product.objects.get_or_create(
        seller=seller,
        name=name,
        defaults={"category": categories[slug], "price": price,
                  "stock": stock, "description": description},
    )
```

- `products` 是有順序的 list；每筆商品資料是固定欄位的 tuple
- `categories` 是 dict，使用 `slug` 查到對應的分類物件
- `defaults={...}` 是另一個 dict，將欄位名稱對應到要寫入的值

這是「容器巢狀」的實際樣子：list 裝 tuple，函式參數再接收 dict。

第 3 章 | 動手試與觀念檢核

動手試： 出一個迷你 context 並取值：

```
context = {  
    "query": "鍵盤",  
    "products": [{"name": "機械鍵盤", "price": 350}],  
}  
print(context.get("category", "全部"))  
print(context["products"][0]["name"])
```

觀念檢核：

1. list 和 tuple 最大的差別？ **答：tuple 唯讀、list 可增改**
2. `{"q": "x"}["cat"]` 會怎樣？ **答：KeyError**
3. `.get("cat", "")` 呢？ **答：回空字串，不丟例外**
4. Template context 傳的是哪種容器？ **答：dict**

第 4 章

流程控制與真偽值

本章成果： 能寫 if/elif/else 分支，並用 truthy/falsy 解釋 `request.POST or None` 這類慣用法。

4-1 if / elif / else : 縮排就是語法

```
quantity = 3

if quantity <= 0:
    print("數量必須為正")
elif quantity > stock:
    print("庫存不足")
else:
    print("可以加入購物車")
```

- 冒號 `:` 開啟區塊；**縮排 4 格**定義隸屬關係，不是美觀
- 同一層區塊縮排必須一致

常見錯誤： `IndentationError: expected an indented block`——冒號後忘記縮排。

4-2 比較與邏輯運算

運算	寫法	例子
等於 / 不等於	<code>==</code> / <code>!=</code>	<code>request.method == "POST"</code>
大小比較	<code><</code> <code><=</code> <code>></code> <code>>=</code>	<code>quantity > product.stock</code>
且 / 或 / 非	<code>and</code> <code>or</code> <code>not</code>	<code>if q and category:</code>

```
if request.method == "POST" and form.is_valid():  
    ...
```

常見錯誤： `=` 是綁定、`==` 才是比較。 `if x = 5:` 直接 `SyntaxError`。

4-3 Truthy / Falsy：每個值都能放進 if

Falsey (當成 False)	Truthy (當成 True)
<code>False</code> 、 <code>None</code>	其他一切
<code>0</code> 、 <code>0.0</code>	非零數字
<code>""</code> 空字串	非空字串
<code>[]</code> <code>{}</code> 空 list/dict/set	有元素的容器

```
query = ""  
if query:  
    ... # 不執行：空字串是 falsey
```

LearnMart 對照（重要伏筆）：

```
form = CheckoutForm(request.POST or None)
```

空的 POST QueryDict 是 **falsey** → `or None` 生效 → form 變成 unbound。

4-4 `is None` vs `== None`

```
if average is None:      # 推荐：判斷「同一個物件 None」  
if average == None:      # 能動，但不合慣例
```

- 判斷 `None` 用 `is`；判斷值相等用 `==`
- 反過來的寫法也常見：`if query is not None:`

口訣：**`is` 問身份**，**`==` 問內容**。

目前專案盤點：兩個專案會回傳 `None`，但 Python 原始碼目前沒有直接寫 `if value is None`；常見的是 `if self.author`、`if result` 或 `if not items`。這些寫法是在判斷 `truthy/falsy`，並不只專指 `None`。

4-5 Guard clause：先擋掉例外情況

```
def register(request):  
    if request.user.is_authenticated:      # 先處理「不在狀況內」的  
        return redirect("marketplace:home")  
    # 以下才是主流程  
    ...
```

先 return 掉特殊情況 → 主流程不用包在多層 else 裡，可讀性高。

這段就是兩個專案 `register()` view 的真實開頭——LearnBoard 的 `board/views.py` 與此幾乎一模一樣。

4-6 LearnBoard：登入狀態與表單驗證的分支

```
# learnboard/board/views.py
def register(request):
    if request.user.is_authenticated:
        return redirect("board:list")
    form = RegistrationForm(request.POST or None)
    if request.method == "POST" and form.is_valid():
        user = form.save()
        login(request, user)
        return redirect("board:list")
```

- 第一個 `if` 是 guard clause：已登入就直接離開，不繼續建立註冊表單
- `request.POST or None` 使用 `truthy/falsy` 表達 GET 與 POST 的差異
- `and` 要求兩個條件都成立，才會儲存表單並登入

這不是把所有程式塞進 `else`；先處理例外狀況，主流程會比較平坦。

4-7 LearnMart：購物車規則就是連續的分支

```
# learnmart/marketplace/views.py
if quantity > product.stock:
    messages.error(request, "加入數量超過目前庫存。")
else:
    item, created = CartItem.objects.get_or_create(...)
    item.quantity = quantity if created else min(
        item.quantity + quantity, product.stock
    )
```

- `>` 判斷數量是否超過庫存
- `if ... else ...` 決定顯示錯誤，或更新購物車
- 條件運算式 `quantity if created else ...` 依 `created` 的真假選值

結帳 view 另有 `if not items:`：購物車是空容器時，直接回到購物車頁面。

第 4 章 | 動手試與觀念檢核

動手試： 寫一個庫存判斷：

```
stock, qty = 5, 3
if qty > stock:
    print("庫存不足")
else:
    stock -= qty
print(stock)      # ?
```

觀念檢核：

1. `bool("")` ? 答：False
2. `bool([0])` ? 答：True（非空容器，元素是什麼不重要）
3. `if x = 3:` 會怎樣？ 答：SyntaxError
4. guard clause 解決什麼問題？ 答：減少巢狀、主流程更清楚

第 5 章

迴圈：for 與它的朋友們

本章成果： 能用 for 走訪容器，讀懂 comprehension 與 `sum(... for ...)` 寫法。

5-1 for：走訪每個元素

```
for name in ["鍵盤", "筆記本", "滑鼠"]:  
    print(f"- {name}")
```

- 鍵盤
- 筆記本
- 滑鼠

- for 後面接任何「可走訪」的東西：list、str、dict...
- Django 模板的 `{% for product in products %} ... {% endfor %}` 就是同一件事

5-2 range 與 enumerate

```
for i in range(3):           # 0, 1, 2 (不含 3)
    print(i)

for i, name in enumerate(["a", "b"]): # 同時要索引時
    print(i, name)
# 0 a
# 1 b
```

需要「第幾個 + 內容」就用 `enumerate`，不要自己養計數器。

專案盤點： LearnMart 的表單使用 `range(1, 6)` 產生 1~5 顆星；兩個專案目前沒有直接使用 `enumerate()`。

5-3 break、continue 與 while

```
for item in items:
    if item.quantity <= 0:
        continue      # 跳過這個，繼續下一個
    if item.product.stock == 0:
        break          # 整個迴圈直接結束

while stock > 0:      # 條件成立就一直跑
    ...
```

Web 程式**九成**用 for；while 在「倒扣直到歸零」這類場景才出現。

5-4 Comprehension 串流解析(生成)

```
prices = [350, 60, 500]
doubled = [p * 2 for p in prices]           # [700, 120, 1000]
expensive = [p for p in prices if p >= 300] # [350, 500]

names = [p["name"] for p in products]      # 從 dict 抽欄位
```

讀法：「對 prices 裡每個 p，產生一個結果（可用 if 篩選）」。

進階形：

```
total = sum(item.subtotal for item in items)
```

沒有方括號的版本叫 generator expression，專門餵給 `sum()` 這類函式。

5-5 LearnBoard：for 走訪表單欄位與示範資料

```
# learnboard/board/forms.py
for field in self.fields.values():
    css_class = "form-select" if isinstance(field.widget, forms.Select) else "form-control"
    field.widget.attrs.setdefault("class", css_class)
```

```
# learnboard/board/management/commands/seed_demo.py
for username, content in samples:
    Message.objects.get_or_create(
        content=content,
        defaults={"author": users[username] if username else None},
    )
```

- 第一個迴圈逐一處理每個表單欄位
- 第二個迴圈逐一處理 `samples` 裡的 tuple，並解包成 `username`、`content`
- 迴圈內的 `if ... else ...` 決定訪客留言是否有作者

5-6 LearnMart : list comprehension 與 generator expression

```
# learnmart/marketplace/forms.py
choices = [(number, "★" * number) for number in range(1, 6)]
```

```
# learnmart/marketplace/views.py
total = sum(item.subtotal for item in items)
```

- 第一行是 list comprehension：每個 `number` 產生一組 (值, 星號文字) tuple
- `sum(item.subtotal for item in items)` 沒有方括號，是 generator expression
- generator 逐個提供小計給 `sum()`，不用先建立另一個完整 list

5-7 目前專案沒有使用的迴圈控制

目前 LearnBoard／LearnMart 的 Python 原始碼中：

- 有 `for`、`range()`、list comprehension 與 generator expression
- 沒有直接使用 `while`、`break`、`continue` 或 `enumerate()`

這些語法仍要會讀，因為它們是一般 Python 工具；但不要在程式碼盤點中把教材練習範例誤認為兩個專案目前的實作。

第 5 章 | 動手試與觀念檢核

動手試：

```
cart = [  
    {"name": "鍵盤", "qty": 1, "price": 350},  
    {"name": "筆記本", "qty": 3, "price": 60},  
]  
subtotals = [i["qty"] * i["price"] for i in cart]  
print(sum(subtotals))      # ? (530)
```

觀念檢核：

1. `range(2, 5)` 產生哪幾個數？ 答：2, 3, 4
2. `continue` 和 `break` 差在哪？ 答：跳過本次 vs 結束整個迴圈
3. `[x["name"] for x in cart]` 回傳什麼型別？ 答：list

第 6 章

函式：def、參數與 `*args` / `**kwargs`

本章成果：能定義函式、分辨參數形式，讀懂 view function 與 `get_context_data(self, **kwargs)`。

6-1 定義與呼叫

```
def greet(name):           # def 名字(參數):  
    return f"Hi, {name}"   # return 把結果交回呼叫處  
  
message = greet("leon")    # 呼叫 ; message == 'Hi, leon'
```

- 函式名用 snake_case，用下底線連結、全字母小寫。例如 hello_world_2026()
- 沒寫 `return`（或只寫 `return`）→ 回傳 `None`

```
def show(x):  
    print(x)  
  
result = show(1)           # result 是 None ! 印東西 ≠ 回傳值
```

6-2 參數 vs 引數；位置 vs 關鍵字

```
def render_page(template, context):    # 參數：定義時的名字
    ...

render_page("home.html", {})          # 引數依位置對應
render_page(context={}, template="home.html") # 關鍵字引數：指名道姓
```

LearnMart 對照：

```
render(request, "marketplace/home.html", {"products": products})
#      位置引數  位置引數                位置引數 (dict)
path("products/<int:pk>/", views.product_detail, name="product-detail")
```

關鍵字引數可讀性高，Django 的 `path(name=...)` 就是範例。

6-3 預設值參數

```
def search(query, category=""):      # category 不傳就用 ""
    ...

search("鍵盤")                      # 等同 search("鍵盤", "")
search("鍵盤", category="3c")      # 想給就給
```

LearnMart 對照： `request.GET.get("q", "")`——`get` 方法的第二個參數就是預設值。

預設值放在參數清單後段；有預設的不能排在無預設前面。

6-4 *args : 收成一個 tuple

```
def total(*numbers):           # numbers 是 tuple，裝下所有位置引數
    return sum(numbers)
```

```
total(10, 20, 30)             # 60
```

- * 在**定義**端＝打包；在**呼叫**端＝拆包
- 打包後就是一個普通 tuple，能用 for 走訪

6-5 ****kwargs** : 收成一個 dict

```
def build_context(**extra):          # extra 是 dict, 裝下所有關鍵字引數
    print(extra)

build_context(page=2, q="鍵盤")
# {'page': 2, 'q': '鍵盤'}
```

LearnMart 對照：

```
def get_context_data(self, **kwargs):
    context = super().get_context_data(**kwargs)
    ...
```

父類不知道子類會多傳什麼 → 用 ****kwargs** 全收再轉傳。看到 ****kwargs** 就想成「彈性行李箱」。

6-6 作用域：函式內有自己的世界

```
count = 0                # 全域

def add():
    count = 99            # 這是函式內的區域變數，不影響外面的 count
    return count

add()                     # 99
print(count)              # 0 (全域沒被改)
```

- 函式內可以**讀**外面的名字；指定值則會創造**區域新變數名稱**
- 需要結果就 return，不要靠改全域

Django view 幾乎不用全域變數——每個 request 都是獨立函式呼叫。

6-7 LearnBoard：同一個專案同時有 function view 與 method

```
# learnboard/board/views.py
def register(request):
    if request.user.is_authenticated:
        return redirect("board:list")
    form = RegistrationForm(request.POST or None)
    return render(request, "registration/register.html", {"form": form})

class MessageListView(ListView):
    def get_queryset(self):
        queryset = Message.objects.select_related("author")
        return queryset
```

- `register(request)` 是直接呼叫的 view function
- `get_queryset(self)` 是掛在 `MessageListView` instance 上的方法
- `self` 讓方法取得目前物件；`request` 則是由 Django 傳入的 request

兩者都遵守「接收資料 → 做處理 → `return` 結果」的函式心智模型。

6-8 LearnMart : `*args` / `**kwargs` 是框架的彈性介面

```
# learnmart/marketplace/forms.py
def __init__(self, *args, **kwargs):
    super().__init__(*args, **kwargs)
    self.apply_bootstrap()

# learnmart/marketplace/views.py
def get_context_data(self, **kwargs):
    context = super().get_context_data(**kwargs)
    context["categories"] = Category.objects.all()
    return context
```

- `*args` 收集位置引數，形成 tuple
- `**kwargs` 收集關鍵字引數，形成 dict
- 呼叫 `super()` 時再用 `*` / `**` 拆回去，讓父類收到原本的參數

6-8B LearnMart : `*args` / `**kwargs` 是框架的彈性介面 (續)

這種寫法讓 Django 可以在不固定參數數量的情況下，繼續傳遞表單與 context 的資料。

你不必一次記住所有 Django 類別的細節；先記住：框架會幫你包裝與拆解參數，程式碼看起來雖然不固定，但邏輯其實很穩定。

第 6 章 | 動手試與觀念檢核

****動手試：** **寫一個迷你 subtotal 函式：

```
def subtotal(price, qty, discount=0):  
    return price * qty - discount  
  
print(subtotal(350, 2))          # ?  
print(subtotal(350, 2, discount=100)) # ?
```

觀念檢核：

1. 沒寫 return 的函式回傳？ **答：None**
2. `def f(a, b="x")` 可以改成 `def f(a="x", b)` 嗎？ **答：不行**
3. `**kwargs` 收到的型別？ **答：dict**
4. `super().method(**kwargs)` 中第二個 `**` 是在做什麼？ **答：呼叫端的拆包**

第 7 章

例外處理：try / except

本章成果： 能讀 traceback、用 `try/except ValueError` 保護轉型，並理解 `raise`。

7-1 讀懂 traceback：從最後一行往上看

```
Traceback (most recent call last):  
  File "scratch.py", line 4, in <module>  
    qty = int("三個")  
ValueError: invalid literal for int() with base 10: '三個'
```

閱讀順序：

1. 最後一行：例外種類 + 訊息（`ValueError: ...`）
2. 往上找「你的檔案」那幾行定位行號
3. 中間的框架是函式呼叫路徑

7-2 try / except : 預期中的失敗就接住

```
raw = request.POST.get("quantity", "1")
try:
    quantity = int(raw)
except ValueError:
    quantity = 1          # 使用者送了亂七八糟的字串 → 給安全預設
```

LearnMart 對照： `add_to_cart()` 與 `update_cart()` 都這樣保護 `int()`。

原則：

- 只 catch 你**預期會發生**的例外種類
- 不要裸寫 `except:` 吞掉所有錯誤——除錯時你會感謝當初沒吞

常見錯誤：

```
except:
    pass          # 太寬，連打錯字 NameError 也一起吞
                  # pass 又什麼都不做 = 靜默失敗，最難抓
```

7-3 raise：主動拋出

```
def withdraw(balance, amount):  
    if amount > balance:  
        raise ValueError("餘額不足")  
  
try:  
    withdraw(100, 500)  
except ValueError as e:  
    print(e)                # 餘額不足
```

- `raise 例外類型("訊息")` 中斷流程並向上回報
- `as e` 把例外物件接起來讀訊息

LearnMart 對照概念： checkout 沒有手寫 `raise`；若 ORM 或其他程式錯誤向外拋出，`transaction.atomic` 才能讓交易回滾（Deck 02 第 5 章）。

7-4 LearnMart：把瀏覽器字串轉成可計算的數字

```
# learnmart/marketplace/views.py
try:
    quantity = max(1, int(request.POST.get("quantity", 1)))
except ValueError:
    quantity = 1
```

- `request.POST.get(...)` 取得的輸入來自瀏覽器，不能假設一定是數字
- `int("2")` 成功；`int("兩個")` 或 `int("")` 會拋出 `ValueError`
- 例外發生時使用安全預設值 `1`，讓購物車流程可以繼續

`update_cart()` 也有同樣的轉型保護；這是「輸入邊界先驗證」的實際例子。

7-5 LearnBoard：啟動腳本處理 ImportError

```
# learnboard/manage.py
try:
    from django.core.management import execute_from_command_line
except ImportError as exc:
    raise ImportError(
        "Couldn't import Django. Are you sure it's installed?" # 訊息節錄
    ) from exc
```

- `except ImportError as exc` 把原本的例外物件命名為 `exc`
- `raise ... from exc` 重新拋出更有說明性的錯誤，同時保留原始原因
- 這段在 LearnMart 的 `manage.py` 沒有出現；LearnMart 直接 `import` 後執行

專案盤點：兩個 app 目前沒有自行寫 `raise ValueError(...)`；購物車的 `ValueError` 是 `int()` 轉型時由 Python 拋出的。

第 7 章 | 動手試與觀念檢核

動手試：

```
for raw in ["3", "abc", ""]:  
    try:  
        print(int(raw))  
    except ValueError:  
        print("fallback")
```

觀念檢核：

1. `int("abc")` 拋哪種例外？ 答：ValueError
2. 為什麼不要裸 `except:`？ 答：會吞掉非預期錯誤，難除錯
3. traceback 要先讀哪一行？ 答：最後一行（例外種類 + 訊息）

第 8 章

模組與 import

本章成果： 能分辨三種 import 寫法，讀懂兩個專案開頭的每一行 import。

8-1 三種 import 形式

```
import decimal                # 用 decimal.Decimal

from decimal import Decimal   # 直接用 Decimal (最常見)

from django.db import models  # 從套件匯入指定名字
import numpy as np            # 取暱稱 (慣例用)
```

- import 之後才能使用該模組提供的名字
- 標準庫 → 第三方套件 → 本專案，分組排列是常見慣例

LearnMart 對照：

```
from django.db import models
from django.shortcuts import render, redirect, get_object_or_404

from .models import Product, Order    # ← 專案內自己的 models.py
```

8-2A LearnBoard × LearnMart：同一種 import，不同的領域名字

```
# learnboard/board/views.py
from django.shortcuts import redirect, render
from .forms import MessageForm, RegistrationForm
from .models import Message

# learnmart/marketplace/views.py
from django.shortcuts import get_object_or_404, redirect, render
from .forms import BoardPostForm, CheckoutForm, ProductForm, RegistrationForm, ReviewForm
from .models import BoardPost, CartItem, Category, Order, OrderItem, Product, Review
```

- `django.*` 是第三方套件；`.forms`、`.models` 是目前 App 內的相對 import
- LearnBoard 匯入 `Message`；LearnMart 匯入商城需要的多個 model
- import 清單反映功能範圍，不是「匯入越多越好」

8-2 相對 import：那個點是什麼？

```
from .models import Product      # . = 「同一層資料夾」
```

`board/views.py` 與 `board/models.py` 同層（LearnMart 的 `marketplace/` 也是同樣結構）：

```
board/
|—— views.py    ← from .models import Message
|—— models.py   ← 被匯入的一方
|—— forms.py    ← from .models import Message, User
```

- 一個點 = 目前 package；兩個點 = 上一層（少用）
- 忘記寫相對點寫成 `from models import Product` 在 Django app 內會出問題

LearnBoard 對照： `from . import views` 出現在 `board/urls.py`，代表匯入同一個 `board` package 裡的 `views.py`。

LearnMart 對照： `from django.contrib.auth import views as auth_views` 使用 `as` 取別名，避免和 App 自己的 `views` 名稱混淆。

常見錯誤： `ModuleNotFoundError: No module named 'xyz'` 多半是 (1) 打錯名稱 (2) 環境不對 (沒用 `uv run`) (3) 相對/絕對路徑混用。

8-3 標準庫預告：Decimal 與 timezone

```
from decimal import Decimal  
  
price = Decimal("350")          # 金額一律用字串建 Decimal
```

```
from django.utils import timezone  
  
order.shipped_at = timezone.now()
```

- `Decimal("350")` 不用 float 的理由在 Deck 01 4-10 完整說明
- Django 自帶的工具都從 `django.*` 匯入——這就是「框架」的意義

8-4 專案中的標準庫與框架工具

```
# 兩個專案的 config/settings.py
from pathlib import Path
BASE_DIR = Path(__file__).resolve().parent.parent

# learnmart/marketplace/models.py
from decimal import Decimal

# learnmart/marketplace/views.py
from django.utils import timezone
```

- `pathlib.Path` 是 Python 標準庫，用來組合檔案路徑
- `Decimal` 也是標準庫，LearnMart 用它表示訂單金額的預設值
- `timezone` 來自 Django，LearnMart 出貨時用 `timezone.now()` 記錄時間

目前盤點：兩個專案沒有使用教材示範的 `numpy`；那只是說明「取別名」的一般例子，不是本課程的依賴。

第 8 章 | 觀念檢核

1. `from decimal import Decimal` 之後能寫 `decimal.Decimal` 嗎？ 答：不能，只匯入了 **Decimal 這個名字**
2. `from .models import User` 的 `.` 指？ 答：同層 package (marketplace)
3. `ModuleNotFoundError` 先檢查哪三件事？ 答：拼字／環境／路徑形式

第 9 章

類別基礎：class、self 與 `__init__`

本章成果： 能建立類別與物件、定義 method，讀懂 Model class 與巢狀 `class Meta`。

9-1 class 是藍圖，instance 是成品

```
class Product:  
    pass
```

```
p1 = Product()      # 用藍圖蓋出一件成品 (instance)  
p2 = Product()      # 再蓋一件；p1 和 p2 互不相干
```

- class 名用 **PascalCase**，每一個單字的首字母（包含第一個單字）都採用大寫字母
- 例如：UserProfile
- `p1 = Product()` 讀作「呼叫這個 class，產生一個新物件」

LearnMart 的 `Product.objects.get(pk=1)` 回傳的就是一個 instance。

9-2 `__init__` 與 `self`：物件的出生程序

```
class CartItem:
    def __init__(self, name, qty):    # 建立時自動執行
        self.name = name            # 把參數掛到「這個物件自己」身上
        self.qty = qty

item = CartItem("鍵盤", 2)
print(item.name)    # 鍵盤
```

- `self` = 正在運作的那個物件本人
- `item.name` 其實等價於內部呼叫 `CartItem.__init__(item, "鍵盤", 2)` ——`self` 就是第一個被塞進去的 `item`

你應該看到：方法定義的第一個參數永遠是 `self`，但呼叫時不用傳它。

9-3 method：綁在物件上的函式

```
class CartItem:
    def __init__(self, name, qty, price):
        self.name, self.qty, self.price = name, qty, price

    def subtotal(self):
        # 方法也能用 self 拿自己的資料
        return self.price * self.qty

item = CartItem("鍵盤", 2, 350)
print(item.subtotal()) # 700 (注意有括號——它在「呼叫」)
```

LearnMart 對照：

```
class CartItem(models.Model):
    ...
    @property
    def subtotal(self):
        return self.product.price * self.quantity
```

幾乎一樣！差別只在 `@property`
——下一章揭曉。

9-4 `__str__`：決定 print 出來長怎樣

```
class Category:
    def __init__(self, name):
        self.name = name

    def __str__(self):
        return self.name

c = Category("3C")
print(c)           # 3C (沒有 __str__ 會印 <Category object at 0x...>)
```

LearnMart 對照： `Category`、`Product`、`Order` 等 model 定義了 `__str__`，admin 後台與 shell 的列表顯示會使用它；不是每個 model 都必須自行定義。

命名前後雙底線（dunder）的方法由 Python 在特定時機自動呼叫。

9-5 巢狀類別：`class Meta` 與 `TextChoices`

```
class Product(models.Model):  
    ...  
    class Meta:                                # 住在 Product 裡面的設定集  
        ordering = ["-created_at"]  
  
class User(models.Model):  
    class Role(models.TextChoices):  
        BUYER = "buyer", "買家"                # 住在 User 裡面的選項集
```

- 巢狀類別只是「把設定收進所屬類別的命名空間」，不是繼承也不是 instance
- 看到 `Product.Meta.ordering` 這種存取就知道是往裡面找

你應該看到： `Role.BUYER` 的值是 `"buyer"`，顯示文字是 `"買家"`。

9-6 @property：不用括號的聰明屬性

```
class User(models.Model):
    ...
    @property
    def is_seller(self):
        return self.role == self.Role.SELLER

u.is_seller      # True / False — 沒有小括號！
u.is_seller()    # TypeError：它不是普通方法
```

寫法	意義
def total(self)	方法：obj.total()
@property def total(self)	唸起來像屬性：obj.total

LearnMart 全都在用： is_seller 、 average_rating 、 CartItem.subtotal 、 OrderItem.subtotal 。

模板同理： {{ product.average_rating }} 不加括號。

9-7 LearnBoard : Message 也是普通 Python class 加上 Django 能力

```
# learnboard/board/models.py
class Message(models.Model):
    content = models.TextField("留言內容", max_length=500)
    # author、created_at 等欄位省略

    class Meta:
        ordering = ["-created_at"]

    def __str__(self):
        who = self.author.username if self.author else "訪客"
        return f"{who} : {self.content[:20]}"

    def get_absolute_url(self):
        return reverse("board:list")
```

- Message 是 class，content 是欄位
- __str__() 與 get_absolute_url() 是方法
- class Meta 是巢狀 class，提供 Django 設定
- self 代表目前這筆留言（如 self.author）

與 LearnMart 的 Product、Order 結構完全一致，皆為「class + method + Meta」。

9-8 兩個專案的「計算屬性」對照

```
# learnboard/board/models.py
def __str__(self):
    who = self.author.username if self.author else "訪客"
    return f"{who} : {self.content[:20]}"

# learnmart/marketplace/models.py
@property
def subtotal(self):
    return self.product.price * self.quantity
```

- `__str__()` 是 Python 在需要文字表示時自動呼叫的方法
- `@property` 讓 `item.subtotal` 看起來像欄位，但實際上會執行計算
- 兩者都使用 `self` 讀取目前 instance 的資料

第 9 章 | 動手試與觀念檢核

動手試：

```
class Review:
    def __init__(self, rating):
        self.rating = rating

    @property
    def stars(self):
        return "★" * self.rating

r = Review(4)
print(r.stars)      # ?
```

觀念檢核：

1. `__init__` 何時執行？
答：建立 instance 時自動
2. 方法定義第一個參數？
答：self
3. `{{ post.title }}` 對應 Python 哪種語法？
答：屬性存取（或 `@property`）
4. `class Meta` 會變成資料庫表格嗎？
答：不會，它是設定集

第 10 章

繼承、override、`super()` 與 mixin

本章成果： 能讀懂 `User(AbstractUser)` 與 CBV mixin 的繼承鏈，知道 `super()` 在排隊叫誰。

10-1 繼承：拿現成的藍圖再加料

```
class Animal:
    def eat(self):
        return "吃東西"

class Cat(Animal):           # Cat 繼承 Animal : Cat is a Animal
    def meow(self):
        return "喵"

mii = Cat()
mii.meow()                  # 自己的
mii.eat()                   # 爸媽給的：直接可用
```

- 括號裡是被繼承的父類別
- 子類自動擁有父類全部屬性與方法

LearnMart 對照：

```
class User(AbstractUser):    # Django 幫你做好密碼雜湊、權限...  
    role = models.CharField(...)
```

你只寫加料（role），AbstractUser 的登入能力全部繼承。

10-2 override：同名字方法，子類說了算

```
class Animal:
    def sound(self):
        return "..."

class Dog(Animal):
    def sound(self):           # 覆寫父類版本
        return "汪"
```

```
Dog().sound()    # 汪
Animal().sound() # ... (父類不受影響)
```

- 子類定義同名方法 → 以子類為準
- Django 的 `get_queryset()`、`get_context_data()` 全是 override

10-3 super()：先請父類做完，我再補充

```
class LoginView:
    def dispatch(self, request):
        print("1. 檢查登入")
        ...

class MyView(LoginView):
    def dispatch(self, request):
        super().dispatch(request)    # 先執行父類版
        print("2. 我自己的事")      # 再補自己的
```

LearnMart 真實場景 (forms.py) :

```
class ProductForm(BootstrapFormMixin, forms.ModelForm):  
    def __init__(self, *args, **kwargs):  
        super().__init__(*args, **kwargs)    # 讓 Django 先把表單建好  
        self.apply_bootstrap()                # 我才加上 CSS class
```

順序有意義：沒先呼叫 `super().__init__()`，`self.fields` 還不存在。

常見錯誤： override `__init__` 忘記呼叫 `super().__init__()` → 物件初始化不完整，錯誤往往在很遠的地方爆炸。

10-4 多重繼承與 mixin：Django 的日常

```
class SellerRequiredMixin(LoginRequiredMixin, UserPassesTestMixin):
    def test_func(self):
        return self.request.user.isSeller
```

- 一個 class 可以同時繼承多個父類
- **Mixin = 可堆疊的能力包**：每個 mixin 只負責一件小事
- 找方法的順序由左至右：先問 LoginRequiredMixin 有沒有，再問下一個

你會看到	它補上的能力
LoginRequiredMixin	未登入就導去 login
UserPassesTestMixin	依 test_func() 決定放行
ListView / CreateView	整套列表／新增流程

常見錯誤： mixin 放錯邊—— SellerRequiredMixin(ListView) 寫反了括號方向。慣例：**mixin 在前、generic view 在最右**。

10-5 LearnBoard：同樣的 CBV 繼承鏈

```
# learnboard/board/views.py
class MessageCreateView(LoginRequiredMixin, CreateView):
    model = Message
    form_class = MessageForm

class OwnerOrStaffMixin(LoginRequiredMixin, UserPassesTestMixin):
    def test_func(self):
        user = self.request.user
        return self.get_object().author_id == user.pk or user.is_staff

class MessageDeleteView(OwnerOrStaffMixin, DeleteView):
    model = Message
```

- `MessageCreateView`：登入限制 + 新增流程
- `OwnerOrStaffMixin`：作者本人或管理員權限
- `MessageDeleteView`：自訂 mixin + 刪除流程

LearnMart 的 `SellerRequiredMixin` 亦同，皆是將規則封裝成 class。

10-6 `super()` 在兩個專案的實際位置

```
# learnboard/board/views.py
def form_valid(self, form):
    form.instance.author = self.request.user
    return super().form_valid(form)

# learnmart/marketplace/views.py
def get_context_data(self, **kwargs):
    context = super().get_context_data(**kwargs)
    context["review_form"] = ReviewForm()
    return context
```

- 子類先補上專案自己的資料，再交給父類完成原本的 Django 流程
- `super()` 不是重新建立一個物件，而是沿著繼承鏈呼叫父類版本
- 同一個觀念也出現在兩個專案的 form `__init__()`

10-7 abstract：為被繼承而設計的類別

```
from django.contrib.auth.models import AbstractUser

class User(AbstractUser):    # AbstractUser 本身不是拿來直接使用的成品
    ...
```

- abstract 類別缺了某些「成品」要素，定位就是藍圖的藍圖
- 直接使用會出錯或不符合框架預期；正確用法是繼承它

LearnMart 對照： `config/settings.py` 的 `AUTH_USER_MODEL = "marketplace.User"` 就是在宣告「本專案的 user 是這個繼承成品」。

第 10 章 | 動手試與觀念檢核

動手試：

```
class Base:
    def label(self):
        return "base"

class Child(Base):
    def label(self):
        return super().label() + "+child"

print(Child().label())    # ?
```

觀念檢核：

1. 子類沒定義的方法能呼叫嗎？ 答：能，用父類的
2. override 後想保留父類行為要靠？ 答：super()
3. mixin 通常放在繼承清單的哪一側？ 答：前面(左邊)
4. 為什麼 User 要繼承 AbstractUser 而不是從零寫？ 答：重用密碼/session/權限等安全實作

第 11 章

decorator：@ 開頭的那一行

本章成果： 能解釋 `@require_POST`、`@login_required` 做了什麼，不再把 `@` 當裝飾花紋。

11-1 函式是物件：可以傳遞、可以代管

```
def hello():  
    return "hi"
```

```
f = hello          # 函式也能綁到名字上（沒有小括號！）  
f()                # 'hi' —— 透過新名字照樣呼叫
```

- 加括號＝呼叫；不加括號＝把函式本身當值傳遞
- 這是理解 decorator 的唯一前置知識

11-2 @ 的真面目：包一層再還給你

```
def shout(func):                # 收一個函式
    def wrapper():
        result = func()
        return result.upper()
    return wrapper              # 回傳「加料版」函式

@shout
def greet():
    return "hi"

greet()    # 'HI'
```

@shout 完全等價於：

```
greet = shout(greet)
```

- @ 只是上面那行的漂亮寫法
- 原本的功能還在，只是被包了一層前後處理

11-3 用這個視角讀 LearnMart 的 decorator

```
@require_POST          # 包一層：method != POST → 直接回 405，根本不進函式
@login_required        # 包一層：未登入 → 導向 login 頁
def add_to_cart(request, pk):
    ...                # 走到這裡時，「該擋的」已經擋完
```

decorator	幫你擋掉
@login_required	匿名使用者
@require_POST	GET 等其他 method（防 mutation 被 GET 觸發）
@property	把方法偽裝成屬性

由上而下依序包裹：離 `def` 最近的最後包、最先執行。

第 11 章 | 動手試與觀念檢核

動手試：預測輸出：

```
def trace(func):  
    def wrapper():  
        print("進入")  
        func()  
        print("離開")  
    return wrapper  
  
@trace  
def work():  
    print("工作中")  
  
work()
```

觀念檢核：

1. `@x def f(): ...` 等價於哪一行？ 答：**`f = x(f)`**
2. `@login_required` 沒登入時，view 函式本體會執行嗎？ 答：**不會**
3. `obj.method` 和 `obj.method()` 差在哪？ 答：**取得函式物件 vs 呼叫**

第 12 章

型別標註：`-> QuerySet` 在寫什麼？

本章成果： 能讀懂參數與回傳值的標註語法，並知道它不會被 Python 強制執行。

12-1 基本語法：變數與函式標註

```
name: str = "鍵盤"           # 變數: 型別 = 值
stock: int = 0

def subtotal(price: int, qty: int) -> int:
    return price * qty        # -> 宣告回傳值型別
```

- 標註是給人與工具看的說明文件
- 冒號後放型別； `->` 放在參數清單後宣告回傳型別

12-2 常見標註形狀

標註	意義
list[int]	裝 int 的 list
dict[str, object]	key 是 str 的 dict (context 就長這樣)
str None	字串或 None
QuerySet[Product]	裝 Product 的 QuerySet
**kwargs: object	kwargs 的值型別

LearnMart 投影片出現過的例子：

```
def get_queryset(self) -> QuerySet:
def get_context_data(self, **kwargs) -> dict:
```

str | None 的 | 是「聯集」：兩種可能都接受。

12-3 標註不被執行強制

```
def double(x: int) -> int:  
    return x * 2
```

```
double("ab")    # 'abab'——完全合法！標註不是驗證
```

- Python 仍是動態定型別；標註錯誤要靠外部工具（mypy、pyright）靜態檢查
- 表單／model validation 才是真正擋資料的地方（Deck 01、02 主題）

心智模型：**標註＝註解的正式版**；validation＝執行時的關卡。

第 12 章 | 觀念檢核

1. `def f() -> None:` 表示？ 答：不回傳有效值（或只 return）
2. 標註違反時 Python 會擋下嗎？ 答：不會，需外部工具
3. `str | None` 怎麼念？ 答：str 或 None

第 13 章

對照總表與自我檢查

本章成果： 把十三個章節收斂成一張「看到語法 → 回哪一章查」的地圖。

語法 → 專案對照總表 (1/2)

語法	章節	出現處 (LearnBoard／LearnMart)
f-string	2	<code>messages.success(f"...")</code>
<code>.get(key, 預設)</code>	3	<code>request.GET.get("q", "")</code> (兩個專案同款)
truthy/falsy	4	<code>request.POST</code> or <code>None</code>
comprehension / sum	5	<code>total = sum(item.subtotal for item in items)</code>
預設值參數	6	<code>.get()</code> 、form 初始化
<code>*args</code> <code>**kwargs</code>	6	<code>get_context_data(self, **kwargs)</code>
try/except ValueError	7	<code>int(request.POST.get(...))</code>
相對 import	8	<code>from .models import Message</code> ／ <code>Product</code>

語法 → 專案對照總表 (2/2)

語法	章節	出現處 (LearnBoard／LearnMart)
<code>__init__</code> / self / method	9	所有 model class
<code>class Meta</code> / TextChoices	9	<code>ordering</code> 、 <code>Role.choices</code>
<code>@property</code>	9	<code>is_seller</code> 、 <code>subtotal</code>
繼承 + <code>super()</code>	10	forms.py 的 mixin 組合
mixin 順序	10	<code>SellerRequiredMixin(...)</code>
decorator	11	<code>@login_required</code>
型別標註	12	投影片節錄、官方文件

標示為 LearnBoard 的語法第一階段就會用到；其餘在第二階段商城回收。

自我檢查：開始 Deck 01 前，你應該能……

- [] 用 REPL 測試一段小程式並解讀 traceback 最後一行
- [] 解釋 `dict.get("k", "")` 與 `dict["k"]` 的差異
- [] 寫出一個含預設值參數與回傳值的函式
- [] 說明 `self` 在 `__init__` 與 method 裡的角色
- [] 讀懂 `class User(AbstractUser)` 加了哪些料、繼承了哪些能力
- [] 看到 `@require_POST` 能說出「包一層先擋 method」
- [] 看到 `-> QuerySet` 知道那是回傳值標註而非魔法

全部打勾 → 進入 [HTML/CSS 先備](#)，然後開始 [Deck 01：兩個 Django 專案的共通基礎](#)。

配套指令複習：

```
uv run python          # REPL 練習
uv run python scratch.py # 執行練習檔
```